



K. J. Somaiya School of Engineering,

Mumbai-77

Batch: A-4 Roll No.: 16010122151

Experiment / Assignment / Tutorial No: 5

TITLE : ATAM - Architecture Trade-off Analysis Method for B.Tech Project.

AIM : To apply Human-centric analysis process ATAM for identifying risks early on in software design for the B. Tech Project.

Expected OUTCOME of Experiment: Analyze software architecture using analysis techniques

Books/ Journals/ Websites referred:

1 “Software Architecture, Richard N Taylor etl, Wiley

Theory:

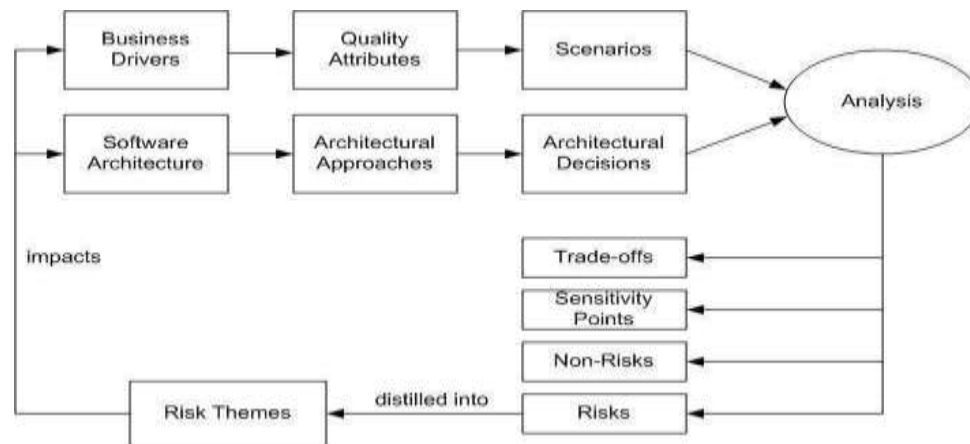
ATAM stands for architectural trade-off analysis method. This method Focuses specifically on four quality attributes (NFPs)

1. Modifiability
2. Security
3. Performance
4. Reliability

Department of Computer Engineering

ATAM reveals how well an architecture satisfies quality goals and how those goals trade-off.

ATAM Process:



The following scenarios can be used:

Use-case scenarios

- Describe how the system is envisioned by the stakeholders to be used

Growth scenarios

- Describe planned and envisioned modifications to the architecture

Exploratory scenarios

- Try to establish the limits of architecture's adaptability with respect to system's functionality, operational profiles



,underlying execution platforms

- Scenarios are prioritized based on importance to stakeholders.

ATAM in a nutshell:

Goals	Completeness
	Consistency
	Compatibility
	Correctness`
Scope	Subsystem- and system-level
	Data exchange
Concern	Non-functional
Models	Informal
	Semi-formal
Type	Scenario-driven
Automation Level	Manual
Stakeholders	Architects
	Developers
	Managers
	Customers

ATAM Analysis of B.Tech Project:



Project Title: ScholarBridge – Scholarship Management System (Student–Trust–Admin portals)

Business Drivers

- **Primary Goal:** Deliver a dependable, secure, and intuitive system enabling students to submit scholarship applications, trusts to evaluate and approve them, and administrators to oversee and address operational concerns.
- **Business Constraints:** The solution must securely manage sensitive personal information (Aadhaar, PAN, banking data), function efficiently on budget smartphones with limited internet access, and support future integration of additional verification mechanisms.
- **Stakeholders:** Includes Students, Trusts (financial sponsors), SuperAdmin/Admin roles, University personnel (verifiers), and the technical team (Developers/Architects).

Software Architecture

Proposed Architecture: Layered Modular Architecture

- **Presentation Layer:** Interfaces for students, trusts, and administrators accessible via web and mobile platforms.
- **Business Logic Layer:** Core backend functionalities such as application handling, verification modules, and payment/disbursement operations.
- **Data Access Layer:** Utilizes a relational database for transactional records and object-based storage for handling uploaded documents.
- **External Services Layer:** Integrates with Aadhaar and PAN verification APIs, messaging services (Email/SMS), and banking systems for fund transfers.

Quality Attributes

- **Security:** Protect personally identifiable information and financial records through robust confidentiality and integrity measures.
- **Reliability:** Ensure consistent and error-free execution of payment and receipt activities.
- **Performance & Scalability:** Capable of managing high traffic during peak periods like application deadlines and trust onboarding phases.
- **Modifiability/Extensibility:** Designed to accommodate new verification rules or APIs with minimal disruption.
- **Usability:** Optimized for basic devices, offering straightforward navigation and support for offline or resumable uploads.
- **Auditability:** Maintains detailed logs tracking each transition in the application lifecycle (e.g., submission → approval → disbursement).

Architectural Approaches

Approach 1: Modular Layered Architecture

- Segregates the system into distinct layers for interface, logic, data, and external integrations.



- A centralized backend enhances control over sensitive operations, ensuring both security and reliability.
- Well-suited for the structured interactions among students, trusts, and administrators, and aligns with regulatory requirements.

Approach 2: Decentralized Peer-to-Peer Model (exploratory)

- Facilitates direct exchange of data and documents between students and trusts.
- Operates without a central server.
- **Why it doesn't fit:**
 - o Decentralized architecture cannot guarantee consistent handling of financial transactions.
 - o Fails to meet security and regulatory standards due to distributed trust data.
 - o Not ideal for low-spec devices, leading to poor user experience.

Analysis

• Performance & Scalability:

- o The layered architecture supports horizontal scaling and queuing mechanisms for disbursement tasks.
- o While P2P may scale in theory, it lacks reliability for accurate financial operations.

• Reliability:

- o Centralized architecture ensures dependable operations with audit trails and system redundancy.
- o P2P setup is vulnerable to disruptions if any participant becomes unavailable.

Trade-offs

- **Simplicity vs Flexibility:** The layered approach offers ease of implementation and dependable behavior but sacrifices decentralization.
- **Immediate feedback vs Reliability in payments:** Real-time disbursement offers responsiveness, whereas asynchronous queuing ensures transactional integrity.
- **Security vs Performance:** Implementing encryption and detailed logging may impact speed but is essential for meeting compliance standards.

Sensitivity Points

- Payment processing mechanisms (ensuring repeat-safe operations and retry logic).
- Verification APIs (potential delays or outages).
- Uploading documents (challenges on low-end devices).
- Role-based access controls (risk of data exposure if misconfigured).



Risks

- **External API downtime (Aadhaar, PAN):** Addressed through retry logic and fallback mechanisms.
- **Data breach:** Mitigated by encryption, role-based access control, and regular security audits.
- **Poor mobile UX:** Countered with simplified forms and support for resumable uploads.

Non-Risks

- Using a relational database for financial records is a robust and suitable choice.
- Document storage in object repositories is secure when protected with encryption and access controls.

Conversion to Risk Themes

- **External API downtime (Aadhaar, PAN):** Solution involves eliminating reliance on third-party APIs by developing proprietary alternatives.
- **Data breach:** Addressed through application-level encryption and document hashing techniques.
- **Poor mobile UX:** Resolved by implementing lightweight input forms and enabling resumable upload functionality.

Post Lab Descriptive Questions

1 . What are the other methods of NFP analysis

In addition to ATAM, several widely used NFP and architectural evaluation techniques include:

- **SAAM (Software Architecture Analysis Method)** — employs scenario-based analysis emphasizing modifiability and functional distribution.
- **CBAM (Cost-Benefit Analysis Method)** — builds upon ATAM by incorporating economic factors to guide architectural prioritization.
- **TARA (Threat Assessment & Remediation Analysis)** — focuses on identifying and mitigating security risks.
- **Architecture Tradeoff Analysis** — includes general adaptations and structured checklists such as those found in **ISO/IEC 25010**, which outlines a comprehensive quality model.
- **Performance modeling and load testing** — utilizes profiling and queuing models to evaluate performance-related NFRs.